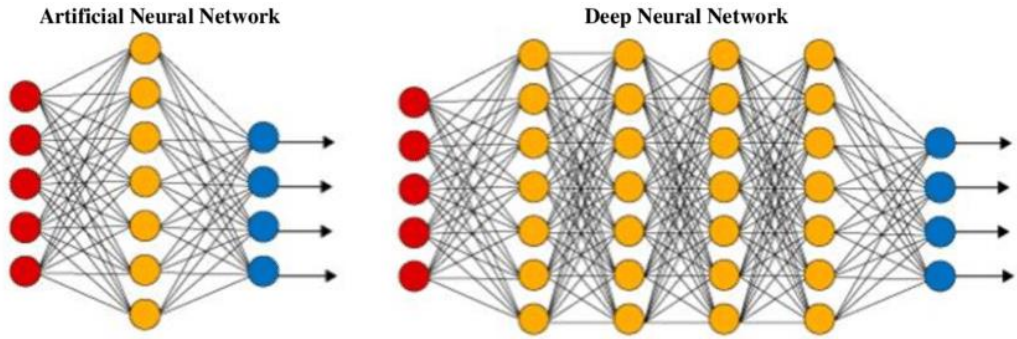
Unit – III

1. Give Introduction to Deep Neural Networks?

A deep neural network (DNN) is an ANN with multiple hidden layers between the input and output layers. Similar to shallow ANNs, DNNs can model complex non-linear relationships.

These networks usually consist of an input layer, one to two hidden layers, and an output layer. While it is possible to solve easy mathematical questions, and computer problems, including basic gate structures with their respective truth tables, it is tough for these networks to solve complicated image processing, computer vision, and natural language processing tasks.

For these problems, we utilize **deep neural networks**, which often have a complex hidden layer structure with a wide variety of different layers, such as a convolutional layer, max-pooling layer, dense layer, and other unique layers. These additional layers help the model to understand problems better and provide optimal solutions to complex projects. A deep neural network has more layers (more depth) than ANN and each layer adds complexity to the model while enabling the model to process the inputs concisely for outputting the ideal solution.



ANN vs DNN - Image Source

Deep neural networks have garnered extremely high traction due to their high efficiency in achieving numerous varieties of deep learning projects. Explore the differences **between machine learning vs deep learning** in a separate article.

Need of DNN: After training a well-built deep neural network, they can achieve the desired results with high accuracy scores. They are popular in all aspects of deep learning, including computer vision, natural language processing, and transfer learning.

The premier examples of the prominence of deep neural networks are their utility in object detection with models such as **YOLO** (You Only Look Once), language translation tasks with BERT (Bidirectional Encoder Representations from Transformers) models, transfer learning models, such as VGG-19, RESNET-50, efficient net, and other similar networks for image processing projects.

Applications for Deep Learning Software

Image recognition is one of the most common applications for deep learning software. By training a system on a large dataset of images, it can learn to recognize objects within them. This technology is being used in facial recognition systems as well as to enable object detection in autonomous vehicles.

Natural language processing (NLP) is another popular application for deep learning software. NLP allows computers to understand human language and respond appropriately to text or voice commands entered by users. This technology is behind virtual assistants such as Alexa and Siri, as well as many automated customer service agents.

OCR (Optical Character Recognition) is a technology that is capable of recognizing text from an image or video by matching it to stored templates. Businesses can use OCR to process large amounts of data very quickly.

Rossum uses deep learning to expand the capabilities of OCR and provide businesses with a more efficient and accurate method of capturing data from invoices or other documents.

By training a neural network to "read" a document like a human can, Rossum offers AI that is capable of accurately extracting the needed information from various document formats without requiring pre-programmed parameters.

Predictive analytics is another area where deep learning software is often applied. By analyzing past data sets, deep learning systems can identify patterns and trends that can predict future outcomes or behaviors with high accuracy levels.

Marketing professionals sometimes use this technology to target customers more effectively based on their past behavior or preferences. In addition, investors use it to create financial forecasting models that help them make better investment decisions based on historical market performance data.

2) What are Deep Learning Platforms?

A deep learning platform typically consists of several components, such as an artificial intelligence (AI) engine, data management tools, development frameworks for building custom models from scratch or fine-tuning existing ones, and deployment options for deploying trained models into production.

- Data management tools provide an efficient way of managing the datasets used to train deep learning models.
- Development frameworks allow users to easily build custom models from scratch or fine-tune existing ones according to their specific needs.
- Deployment options enable users to deploy their trained models into production environments such as cloud services or on-premise servers.

Deep learning platforms also include libraries of pre-trained models containing ready-to-use neural networks that are ready to deploy without any further training or customization. This makes it easy for developers and researchers to get started quickly without worrying about building their own model from the ground up.

Platforms for deep learning use artificial neural networks to enable digital systems to learn and make decisions based on unstructured, unlabeled data.

Artificial neural networks are composed of interconnected nodes that process information in a similar way as the neurons in the human brain. The nodes are organized into layers, and each layer is responsible for a specific task such as pattern recognition or decision-making.

When these networks are trained using large amounts of data, they can learn to recognize patterns and make decisions without being explicitly programmed. This allows them to process unstructured, unlabeled data and draw accurate conclusions from it.

As deep learning has become more popular and widely used, a number of different platforms have emerged to facilitate its development. The most popular deep learning platforms include:

- **Google's TensorFlow** – ideal for large-scale machine learning projects due to its scalability and flexibility
- **Microsoft's Cognitive Toolkit (CNTK)** – commercial-grade distributed deep learning
- **Amazon Web Services (AWS) Deep Learning AMIs** – optimized for cloud computing
- **Apache MXNet** – supports a wide range of programming languages
- **PyTorch** – designed specifically for research purposes

Each of these deep learning libraries offers unique features and capabilities that are suited for different types of tasks. When comparing deep learning platforms, there are several factors to consider. First, it's important to assess the platform's scalability (how easily it can handle large datasets or complex models) as well as its performance in terms of speed and accuracy.

Additionally, developers should look at the platform's ease of use (how quickly they can get up and running with their project) as well as the availability of deep learning tutorials or other resources that can help them learn how to use the platform effectively.

3) What are Deep Learning Tools and Libraries?

Constructing neural networks from scratch helps programmers to understand concepts and solve trivial tasks by manipulating these networks. However, building these networks from scratch is time-consuming and requires enormous effort. To make deep learning simpler, we have several tools and libraries at our disposal to yield an effective deep neural network model capable of solving complex problems with a few lines of code.

The most popular deep learning libraries and tools utilized for constructing deep neural networks are TensorFlow, Keras, and PyTorch. The Keras and TensorFlow libraries have been linked synonymously since the start of TensorFlow 2.0. This integration allows users to develop complex neural networks with high-level code structures using Keras within the TensorFlow network etc

Deep learning is a subfield of machine learning involving artificial neural networks, which are algorithms inspired by the structure of the human brain. Deep learning has many applications and is used in many of today's AI technologies, such as self-driving cars, news aggregation tools, natural language processing (NLP), virtual assistants, visual recognition, and much more.

In recent years, Python has proven to be an incredible tool for deep learning. Because the code is concise and readable, it makes it a perfect match for deep learning applications. Its simple syntax also enables applications to be developed faster when compared to other programming languages. Another major reason for using Python for deep learning is that the language can be integrated with other systems coded in different programming languages. This makes it easier to blend it with AI projects written in other languages.

1. TensorFlow

TensorFlow is widely considered one of the best Python libraries for deep learning applications. Developed by the Google Brain Team, it provides a wide range of flexible tools, libraries, and community resources. Beginners and professionals alike can use TensorFlow to construct deep learning models, as well as neural networks.

TensorFlow has an architecture and framework that are flexible, enabling it to run on various computational platforms like CPU and GPU. With that said, it performs best when operated on a tensor processing unit (TPU). The Python library is often used to implement reinforcement learning in deep learning models, and you can directly visualize the machine learning models. Some of the main features of TensorFlow:

- Flexible architecture and framework.
- Runs on a variety of computational platforms.
- Abstraction capabilities
- Manages deep neural networks.

2. Pytorch

Another one of the most popular Python libraries for deep learning is Pytorch, which is an open-source library created by Facebook's AI research team in 2016. The name of the library is derived from Torch, which is a deep learning framework written in the **Lua** programming language.

PyTorch enables you to carry out many tasks, and it is especially useful for deep learning applications like NLP and computer vision.

Some of the best aspects of PyTorch include its high speed of execution, which it can achieve even when handling heavy graphs. It is also a flexible library, capable of operating on simplified processors or CPUs and GPUs. PyTorch has powerful APIs that enable you to expand on the library, as well as a natural language toolkit. Some of the main features of PyTorch:

- Statistical distribution and operations
- Control over datasets
- Development of deep learning models
- Highly flexible

3. NumPy

One of the other well-known Python libraries, NumPy can be seamlessly utilized for large multi-dimensional array and matrix processing. It relies on a large set of high-level mathematical functions, which makes it especially useful for efficient fundamental scientific computations in deep learning.

NumPy arrays require a lot less storage area than other Python lists, and they are faster and more convenient to use. The data can be manipulated in the matrix, transposed, and reshaped with the library. NumPy is a great option to increase the performance of deep learning models without too much complex work required. Some of the main features of NumPy:

- Shape manipulation
- High-performance N-dimensional array object
- Data cleaning/manipulation
- Statistical operations and linear algebra

4. Scikit-Learn

Scikit-Learn was originally a third-party extension to the SciPy library, but it is now a standalone Python library on Github. Scikit-Learn includes DBSCAN, gradient boosting, support vector machines, and random forests within the classification, regression, and clustering methods.

One of the greatest aspects of Scikit-Learn is that it's easily interoperable with other SciPy stacks. It is also user-friendly and consistent, making it easier to share and use data. Some of the main features of Scikit-learn:

- Data classification and modeling
- End-to-end machine learning algorithms
- Pre-processing of data
- Model selection

5. SciPy

SciPy is one of the best Python libraries out there thanks to its ability to perform scientific and technical computing on large datasets. It is accompanied by embedded modules for array optimization and linear algebra.

The programming language includes all of NumPy's functions, but it turns them into user-friendly, scientific tools. It is often used for image manipulation and provides basic processing features for high-level, non-scientific mathematical functions. Some of the main features of SciPy:

- User-friendly
- Data visualization and manipulation
- Scientific and technical analysis
- Computes large data sets

6. Pandas

One of the open-source Python libraries mainly used in data science and deep learning subjects is Pandas. The library provides data manipulation and analysis tools, which are used for analyzing data. The library relies on its powerful data structures for manipulating numerical tables and time series analysis.

The Pandas library offers a fast and efficient way to manage and explore data by providing Series and DataFrames, which represent data efficiently while also manipulating it in different ways. Some of the main features Pandas:

- Indexing of data
- Data alignment
- Merging/joining of datasets
- Data manipulation and analysis

7. Microsoft CNTK: Another Python library for deep learning applications is Microsoft CNTK (Cognitive Toolkit), which is formerly known as Computational Network ToolKit. The open-source deep-learning library is used to implement distributed deep learning and machine learning tasks.

CNTK enables you to combine predictive models like convolutional neural networks (CNNs), feed-forward deep neural networks (DNNs), and recurrent neural networks (RNNs), with the CNTK framework. This enables the effective implementation of end-to-end deep learning tasks. Some of the main features of CNTK:

- Open-source
- Implement distributed deep learning tasks
- Combine predictive models with CNTK framework
- End-to-end deep learning tasks

8. Keras: Kears is yet another notable open-source Python library used for deep learning tasks, allowing for rapid deep neural network testing. Keras provides you with the tools needed to construct models, visualize graphs, and analyze datasets. On top of that, it also includes prelabeled datasets that can be directly imported and loaded.

The Keras library is often preferred due to it being modular, extensible, and flexible. This makes it a user-friendly option for beginners. It can also integrate with objectives, layers, optimizers, and activation functions. Keras operates in various environments and can run on CPUs and GPUs. It also offers one of the widest ranges for data types. Some of the main features of Keras:

- Developing neural layers
- Data pooling
- Builds deep learning and machine learning models
- Activation and cost functions

9. Theano: Theano, a numerical computation Python library specifically developed for machine learning and deep libraries. With this tool, you will achieve efficient definition, optimization, and evaluation of mathematical expressions and matrix calculations. All of this enables Theano to be used for the employment of dimensional arrays to construct deep learning models.

Theano is used by a lot of deep learning developers and programmers thanks to it being a highly specific library. It can be used with a graphics processing unit (GPU) instead of a central processing unit (CPU). Some of the main features of Theano:

- Built-in validation and unit testing tools
- High-performing mathematical computations
- Fast and stable evaluations
- Data-intensive calculations

10. MXNet: Closing out our list of the 10 best Python libraries for deep learning is MXNet, which is a highly scalable open-source deep learning framework. MXNet was designed to train and deploy deep neural networks, and it can train models extremely fast.

MXNet supports many programming languages, such as Python, Julia, C, C++, and more. One of the best aspects of MXNet is that it offers incredibly fast calculation speeds and resource utilization on GPU. Some of the main features of MXNet:

- Highly-scalable
- Open-source
- Train and deploy deep learning neural networks
- Trains models fast
- Fast calculation speeds

4) What is Tensorflow for deep learning?

TensorFlow is a powerful and versatile open-source software library for machine learning. It was developed by Google and released under the Apache 2.0 open-source license in 2015. TensorFlow provides an extensive set of APIs for building and training neural networks, as well as tools for deploying models into production systems.

TensorFlow deep learning is used by many large organizations. It's also popular among individual developers who are interested in creating their own applications or exploring the possibilities of artificial intelligence.

When it was first released, TensorFlow quickly became one of the most popular deep learning platforms, with the number of TensorFlow deep learning GitHub commits far surpassing other popular deep learning platforms.

To use TensorFlow, you need to first install it on your computer or server. Once installed, you can start coding with Python using the TensorFlow API to create your own neural networks. You can also access TensorFlow deep learning tutorials from the official website or GitHub repository to get started quickly.

The official TensorFlow GitHub repository contains many TensorFlow deep learning example projects, which can help users get up to speed quickly with how to use the library's API functions and classes correctly when developing their own projects.

There are also plenty of tutorials available online which explain how to build basic models step-by-step, so even those without much experience in AI development can get started quickly with deep learning projects using TensorFlow.

5) Explain about deep learning framework?

Deep learning frameworks enable machines to learn from data in order to make decisions and predictions. These frameworks use neural networks, which are systems of interconnected nodes that can process large amounts of data more efficiently than traditional algorithms.

These frameworks have become increasingly popular over the past few years, and deep learning frameworks in 2023 are expected to continue growing in popularity. Businesses can benefit from deep learning frameworks by using them to automate complex tasks such as customer service, fraud detection, document processing, and more.

For example, businesses can use Rossum's deep learning models to improve optical character recognition (OCR) document data capture processes. OCR is an AI-based technology that uses algorithms to recognize text from scanned documents or images.

OCR, however, is somewhat limited because it relies on manually-programmed parameters that must be re-adjusted for every single document format a business needs to process. Deep learning enables intelligent document processing, which learns how to navigate new formats on its own without human input.

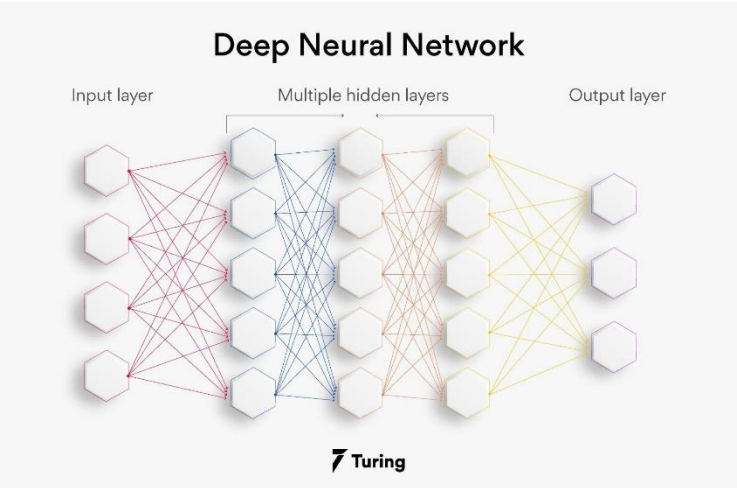
In addition to automating document data capture, businesses can also use deep learning frameworks for predictive analytics. By leveraging the power of AI-driven models, businesses can gain insights into their operations that would be difficult or time-consuming to obtain manually and use these insights to make better strategic decisions.

Ultimately, deep learning frameworks offer a wide range of benefits for businesses looking to leverage AI technologies in order to reduce costs and increase productivity. If you didn't yet take advantage of deep learning frameworks in 2022, they're well worth learning more about in 2023.

6) Explain about Deep feedforward network?

A) Deep feedforward networks, also often called **feedforward neural networks**, or **multilayer perceptrons** (MLPs), are the quintessential deep learning models. The goal of a feedforward network is to approximate some function $\mathbf{f}^*$. For example, for a classifier, $y = \mathbf{f}^*(\boldsymbol{x})$ maps an input $\boldsymbol{x}$ to a category y. A feedforward network defines a mapping $\boldsymbol{y} = \mathbf{f}(\boldsymbol{x}; \boldsymbol{\theta})$ and learns the value of the parameters $\boldsymbol{\theta}$ that result in the best function approximation.

These models are called **feedforward** because information flows through the function being evaluated from $\boldsymbol{x}$, through the intermediate computations used to define $\mathbf{f}$, and finally to the output $\boldsymbol{y}$. There are no **feedback** connections in which outputs of the model are fed back into itself. When feedforward neural networks are extended to include feedback connections, they are called **recurrent neural networks.**



Feedforward networks are of extreme importance to machine learning practioners. They form the basis of many important commercial applications. For example, the convolutional networks used for object recognition from photos are a specialized kind of feedforward network. Feedforward networks are a conceptual stepping stone on the path to recurrent networks, which power many natural language applications.

Feedforward neural networks are called **networks** because they are typically represented by composing together many different functions. The model is associated with a directed acyclic graph describing how the functions are composed together.

For example, we might have three functions $\mathbf{f}^{(1)}$, $\mathbf{f}^{(2)}$, and $\mathbf{f}^{(3)}$ connected in a chain, to form $\mathbf{f}(\boldsymbol{x}) = \mathbf{f}^{(3)}(\mathbf{f}^{(2)}(\mathbf{f}^{(1)}(\boldsymbol{x})))$. These chain structures are the most commonly used structures of neural networks. In this case, $\mathbf{f}^{(1)}$ is called the **first layer** of the network, $\mathbf{f}^{(2)}$ is called the **second layer**, and so on. The overall length of the chain gives the **depth** of the model. It is from this terminology that the name "deep learning" arises. The final layer of a feedforward network is called the **output layer**. During neural network training, we drive $\mathbf{f}(\boldsymbol{x})$ to match $\mathbf{f}^{*}(\boldsymbol{x})$.

The training data provides us with noisy, approximate examples of $\mathbf{f}^{*}(\boldsymbol{x})$ evaluated at different training points. Each example $\boldsymbol{x}$ is accompanied by a label $y \approx \mathbf{f}^{*}(\boldsymbol{x})$. The training examples specify directly what the output layer must do at each point $\boldsymbol{x}$; it must produce a value that is close to y. The behavior of the other layers is not directly specified by the training data. The learning algorithm must decide how to use those layers to produce the desired output, but the training data does not say what each individual layer should do. Instead, the learning algorithm must decide how to use these layers to best implement an approximation of $\mathbf{f}^{*}$. Because the training data does not show the desired output for each of these layers, these layers are called **hidden layers**.

Finally, these networks are called *neural* because they are loosely inspired by neuroscience. Each hidden layer of the network is typically vector-valued. The dimensionality of these hidden layers determines the **width** of the model. Each element of the vector may be interpreted as playing a role analogous to a neuron. Rather than thinking of the layer as representing a single vector-to-vector function, we can also think of the layer as consisting of many **units** that act in parallel, each representing a vector-to-scalar function. Each unit resembles a neuron in the sense that it receives input from many other units and computes its own activation value. The idea of using many layers of vector-valued representation is drawn from neuroscience.

The choice of the functions $\mathbf{f}^{(i)}(\boldsymbol{x})$ used to compute these representations is also loosely guided by neuroscientific observations about the functions that biological neurons compute. However, modern neural network research is guided by many mathematical and engineering disciplines, and the goal of neural networks is not to perfectly model the brain. It is best to think of feedforward networks as function approximation machines that are designed to achieve statistical generalization, occasionally drawing some insights from what we know about the brain, rather than as models of brain function.

One way to understand feedforward networks is to begin with linear models and consider how to overcome their limitations. Linear models, such as logistic regression and linear regression, are appealing because they may be fit efficiently and reliably, either in closed form or with convex optimization. Linear models also have the obvious defect that the model capacity is limited to linear functions, so the model cannot understand the interaction between any two input variables.

To extend linear models to represent nonlinear functions of $\boldsymbol{x}$, we can apply the linear model not to $\boldsymbol{x}$ itself but to a transformed input $\varphi(\boldsymbol{x})$, where φ is a nonlinear transformation.

7) What are advantages and applications of Deep feed forward networks?

A) **Advantages of feed forward Neural Networks**

- Machine learning can be boosted with feed forward neural networks' simplified architecture.

- Multi-network in the feed forward networks operate independently, with a moderated intermediary.
- Complex tasks need several neurons in the network.
- Neural networks can handle and process nonlinear data easily compared to perceptrons and sigmoid neurons, which are otherwise complex.
- A neural network deals with the complicated problem of decision boundaries.
- Depending on the data, the neural network architecture can vary. For example, convolutional neural networks (CNNs) perform exceptionally well in image processing, whereas recurrent neural networks (RNNs) perform well in text and voice processing.
- Neural networks need graphics processing units (GPUs) to handle large datasets for massive computational and hardware performance. Several GPUs get used widely in the market, including Kaggle Notebooks and Google Collab Notebooks.

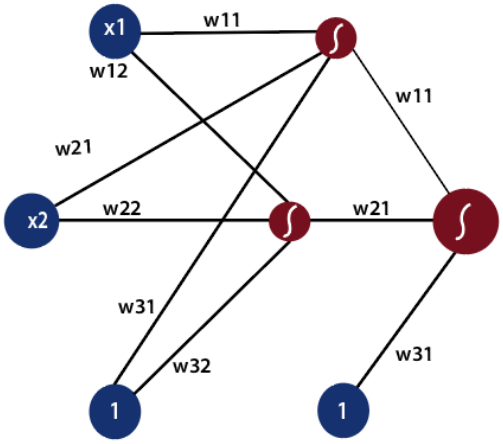
Applications of feed forward neural networks: There are many applications for these neural networks. The following are a few of them.

- **Physiological feed forward system:** It is possible to identify feed forward management in this situation because the central involuntary regulates the heartbeat before exercise.
- **Gene regulation and feed forward:** Detecting non-temporary changes to the atmosphere is a function of this motif as a feed forward system. You can find the majority of this pattern in the illustrious networks.
- **Automation and machine management:** Automation control using feed forward is one of the disciplines in automation.
- **Parallel feed forward compensation with derivative:** An open-loop transfer converts non-minimum part systems into minimum part systems using this technique.

8) Explain about the math behind the Deep Feed Forward networks?

A) "The process of receiving an input to produce some kind of output to make some kind of prediction is known as Feed Forward." Feed Forward neural network is the core of many other important neural networks such as convolution neural network.

In the feed-forward neural network, there are not any feedback loops or connections in the network. Here is simply an input layer, a hidden layer, and an output layer.



There can be multiple hidden layers which depend on what kind of data you are dealing with. The number of hidden layers is known as the depth of the neural network. The deep neural network can learn from more functions. Input layer first provides the neural network with data and the output layer then make predictions on that data which is based on a series of functions. ReLU Function is the most commonly used activation function in the deep neural network.

To gain a solid understanding of the feed-forward process, let's see this mathematically.

1) The first input is fed to the network, which is represented as matrix x1, x2, and one where one is the bias value.

$$[x_1 \quad x_2 \quad 1]$$

2) Each input is multiplied by weight with respect to the first and second model to obtain their probability of being in the positive region in each model.
So, we will multiply our inputs by a matrix of weight using matrix multiplication.

$$[x_1 \quad x_2 \quad 1] = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} = [\text{score} \quad \text{score}]$$

3) After that, we will take the sigmoid of our scores and gives us the probability of the point being in the positive region in both models.

$$\frac{1}{1+e^{-x}}[\text{score score}] = \text{probability}$$

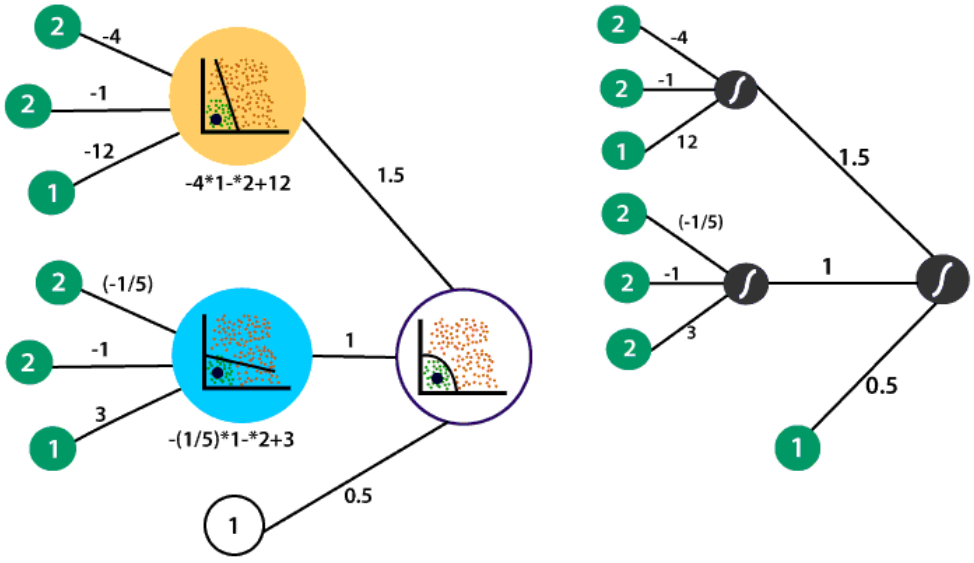
4) We multiply the probability which we have obtained from the previous step with the second set of weights. We always include a bias of one whenever taking a combination of inputs.

$$[\text{probability} \quad \text{probability} \quad 1] \times \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{bmatrix} = [\text{score}]$$

And as we know to obtain the probability of the point being in the positive region of this model, we take the sigmoid and thus producing our final output in a feed-forward process.

$$\frac{1}{1+e^{-x}}[\text{score}] = [\text{probability}]$$

Let takes the neural network which we had previously with the following linear models and the hidden layer which combined to form the non-linear model in the output layer.



So, what we will do we use our non-linear model to produce an output that describes the probability of the point being in the positive region. The point was represented by 2 and 2. Along with bias, we will represent the input as

$$[2 \quad 2 \quad 1]$$

The first linear model in the hidden layer recall and the equation defined it

$$-4x_1 - x_2 + 12$$

Which means in the first layer to obtain the linear combination the inputs are multiplied by -4, -1 and the bias value is multiplied by twelve.

$$[2 \quad 2 \quad 1] \quad \times \begin{bmatrix} -4 & w_{12} \\ -1 & w_{22} \\ 12 & w_{32} \end{bmatrix}$$

The weight of the inputs are multiplied by -1/5, 1, and the bias is multiplied by three to obtain the linear combination of that same point in our second model.

$$[2 \quad 2 \quad 1] \quad \times \begin{bmatrix} -4 & -1/5 \\ -1 & -1 \\ 12 & 3 \end{bmatrix}$$

$$\begin{bmatrix} 2(-4) + 2(-1) + 1(12) \\ 2(-4) + 2(-1) + 1(12) \end{bmatrix}$$

$$[2 \quad 0.6]$$

Now, to obtain the probability of the point is in the positive region relative to both models we apply sigmoid to both points as

$$\left[\frac{1}{1+\frac{1}{e^{x}}} \quad \frac{1}{1+\frac{1}{e^{x}}}\right] = \left[\frac{1}{1+\frac{1}{e^{2}}} \quad \frac{1}{1+\frac{1}{e^{0.6}}}\right] = [0.88 \quad 0.64]$$

The second layer contains the weights which dictated the combination of the linear models in the first layer to obtain the non-linear model in the second layer. The weights are 1.5, 1, and a bias value of 0.5.
Now, we have to multiply our probabilities from the first layer with the second set of weights as

$$[0.88 \quad 0.64 \quad 1] \times \begin{bmatrix} 1.5 \\ 1 \\ 0.5 \end{bmatrix} = [0.88(1.5) + (0.64)(1) + 1(0.5)] = 2.46$$

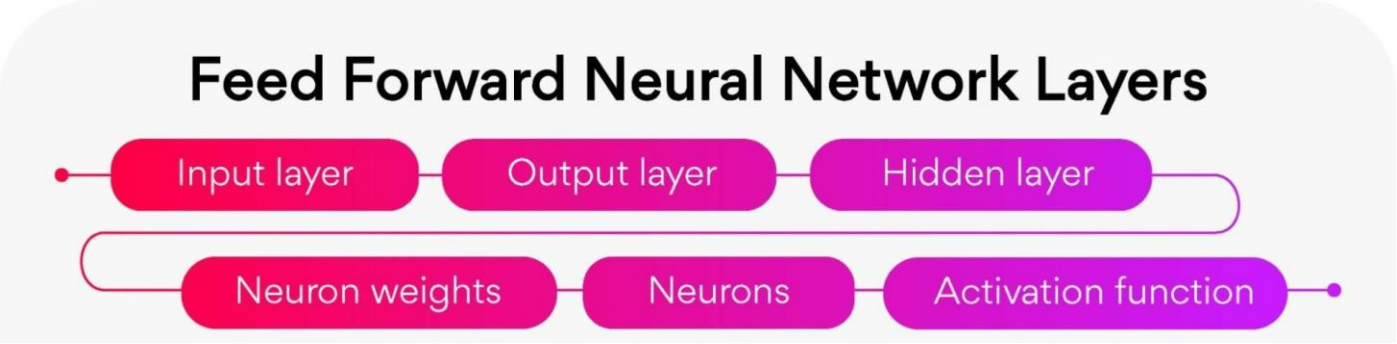
Now, we will take the sigmoid of our final score

$$\frac{1}{1+\frac{1}{e^{2.46}}} = [0.92]$$

It is complete math behind the feed forward process where the inputs from the input traverse the entire depth of the neural network. In this example, there is only one hidden layer. Whether there is one hidden layer or twenty, the computational processes are the same for all hidden layers.

9) Explain about various Layers and functions in Deep Feed forward networks?

A) **Layers of feed forward neural network**



- **Input layer:** The neurons of this layer receive input and pass it on to the other layers of the network. Feature or attribute numbers in the dataset must match the number of neurons in the input layer.
- **Output layer:** According to the type of model getting built, this layer represents the forecasted feature.
- **Hidden layer**: Input and output layers get separated by hidden layers. Depending on the type of model, there may be several hidden layers. There are several neurons in hidden layers that transform the input before actually transferring it to the next layer. This network gets constantly updated with weights in order to make it easier to predict.
- **Neuron weights:** Neurons get connected by a weight, which measures their strength or magnitude. Similar to linear regression coefficients, input weights can also get compared. Weight is normally between 0 and 1, with a value between 0 and 1.
- **Neurons:** Artificial neurons get used in feed forward networks, which later get adapted from biological neurons. A neural network consists of artificial neurons. **Neurons function in two ways:** first, they create weighted input sums, and second, they activate the sums to make them normal. Activation functions can either be linear or nonlinear. Neurons have weights based on their inputs. During the learning phase, the network studies these weights.
- **Activation Function:** Neurons are responsible for making decisions in this area. According to the activation function, the neurons determine whether to make a linear or nonlinear decision. Since it passes through so many layers, it prevents the cascading effect from increasing neuron outputs. An activation function can be classified into three major categories: sigmoid, Tanh, and Rectified Linear Unit (ReLu).

Function in feed forward neural network: The various functions used in deep feed forward networks are

1. Cost function
2. Loss function
3. Gradient Learning
4. Output units

Cost function: In a feed forward neural network, the cost function plays an important role. The categorized data points are little affected by minor adjustments to weights and biases. Thus, a smooth cost function can get used to determine a method of adjusting weights and biases to improve performance. Following is a definition of the mean square error cost function:

$$C(w,b) \equiv \frac{1}{2n} \sum_x \|y(x) - a\|^2.$$

Where,
w = the weights gathered in the network
b = biases
n = number of inputs for training
a = output vectors
x = input
‖v‖ = vector v's normal length

Loss function: The loss function of a neural network gets used to determine if an adjustment needs to be made in the learning process. Neurons in the output layer are equal to the number of classes. Showing the differences between predicted and actual probability distributions. Following is the cross-entropy loss for binary classification.

Cross Entropy Loss:

$$L(\Theta) = \begin{cases} -log(\hat{y}) & \text{if } y = 1 \\ -log(1-\hat{y}) & \text{if } y = 0 \end{cases}$$

As a result of multiclass categorization, a cross-entropy loss occurs:

Cross Entropy Loss:

$$L(\Theta) = -\sum_{i=1}^{k} y_i \log(\hat{y}_i)$$

Gradient learning algorithm: In the gradient descent algorithm, the next point gets calculated by scaling the gradient at the current position by a learning rate. Then subtracted from the current position by the achieved value. To decrease the function, it subtracts the value (to increase, it would add). As an example, here is how to write this procedure:

$$p_{n+1} = p_n - \eta \nabla f(p_n)$$

The gradient gets adjusted by the parameter η, which also determines the step size. Performance is significantly affected by the learning rate in machine learning.

Output units: In the output layer, output units are those units that provide the desired output or prediction, thereby fulfilling the task that the neural network needs to complete. There is a close relationship between the choice of output units and the cost function. Any unit that can serve as a hidden unit can also serve as an output unit in a neural network.

10) Give an example of Learning XOR?

A) To make the idea of a feedforward network more concrete, we begin with an example of a fully functioning feedforward network on a very simple task: learning the XOR function.

The XOR function ("exclusive or") is an operation on two binary values, x_1 and x_2. When exactly one of these binary values is equal to 1, the XOR function returns 1. Otherwise, it returns 0. The XOR function provides the target function $y = f^*(x)$ that we want to learn. Our model provides a function $y = f(x;\theta)$ and our learning algorithm will adapt the parameters θ to make f as similar as possible to f^*.

In this simple example, We want our network to perform correctly on the four points $X = \{[0,0]^T, [0,1]^T, [1,0]^T$, and $[1,1]^T\}$. We will train the network on all four of these points. The only challenge is to fit the training set.

We can treat this problem as a regression problem and use a mean squared error loss function. We choose this loss function to simplify the math for this example as much as possible. In practical applications, MSE is usually not an appropriate cost function for modeling binary data. Evaluated on our whole training set, the MSE loss function is

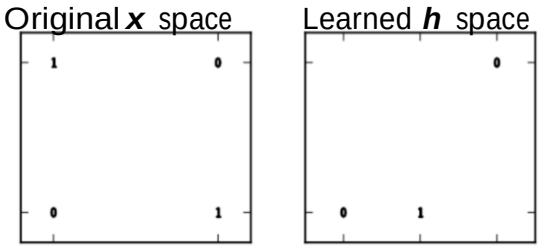
$$J(\theta) = \frac{1}{4}\sum_{x\in X}(f*(x) - f(x;\theta))2$$

Now we must choose the form of our model, $f(x;\theta)$. Suppose that we choose a linear model, with θ consisting of w and b. Our model is defined to be

$$f(x;w,b) = x^T w + b.$$

We can minimize $J(\theta)$ in closed form with respect to w and b using the normal equations.

After solving the normal equations, we obtain $w = 0$ and $b = 1/2$. The linear model simply outputs 0.5 everywhere. The following Figure shows how a linear model is not able to represent the XOR function. One way to solve this problem is to use a model that learns a different feature space in which a linear model is able to represent the solution.



Specifically, we can introduce a very simple feedforward network with one hidden layer containing two hidden units.

This feedforward network has a vector of hidden units h that are computed by a function $f^{(1)}(x; W, c)$. The values of these hidden units are then used as the input for a second layer. The second layer is the output layer of the network. The output layer is still just a linear regression model, but now it is applied to h rather than to x. The network now contains two functions chained together: $h = f^{(1)}(x; W,$

$\boldsymbol{c}$) and $y = \mathbf{f}^{(2)}(\boldsymbol{h}; \boldsymbol{w}, b)$, with the complete model being $\mathbf{f}(\boldsymbol{x}; \boldsymbol{W}, \boldsymbol{c}, \boldsymbol{w}, b) = \mathbf{f}^{(2)}(\mathbf{f}^{(1)}(\boldsymbol{x}))$. The solution we described to the XOR problem is at a global minimum of the loss function.

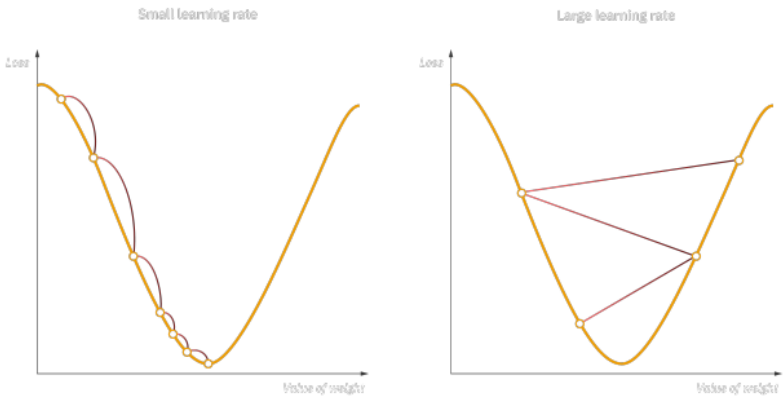
11) Explain about gradient based Learning?

A) **Gradient-based learning** is a type of machine learning in which the optimization algorithm uses gradients to update the model parameters during training. This approach is commonly used in deep learning and neural networks because it allows the model to learn complex representations of the input data.

Gradient Descent: Gradient descent is an optimization algorithm which is commonly-used to train machine learning models and neural networks. Training data helps these models learn over time, and the cost function within gradient descent specifically acts as a barometer, gauging its accuracy with each iteration of parameter updates. Until the function is close to or equal to zero, the model will continue to adjust its parameters to yield the smallest possible error. Once machine learning models are optimized for accuracy, they can be powerful tools for artificial intelligence (AI) and computer science applications.

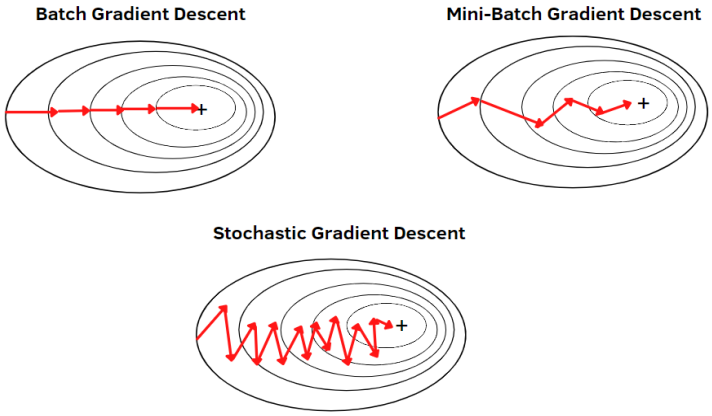
Working of gradient descent: The goal of gradient descent is to minimize the cost function, or the error between predicted and actual y. In order to do this, it requires two data points—a direction and a learning rate. These factors determine the partial derivative calculations of future iterations, allowing it to gradually arrive at the local or global minimum (i.e. point of convergence).

- **Learning rate** (also referred to as step size or the alpha) is the size of the steps that are taken to reach the minimum. This is typically a small value, and it is evaluated and updated based on the behavior of the cost function. High learning rates result in larger steps but risks overshooting the minimum. Conversely, a low learning rate has small step sizes. While it has the advantage of more precision, the number of iterations compromises overall efficiency as this takes more time and computations to reach the minimum.
- **The cost (or loss) function** measures the difference, or error, between actual y and predicted y at its current position. This improves the machine learning model's efficacy by providing feedback to the model so that it can adjust the parameters to minimize the error and find the local or global minimum. It continuously iterates, moving along the direction of steepest descent (or the negative gradient) until the cost function is close to or at zero. At this point, the model will stop learning. Additionally, while the terms, cost function and loss function, are considered synonymous, there is a slight difference between them. It's worth noting that a loss function refers to the error of one training example, while a cost function calculates the average error across an entire training set.



Types of gradient descent: There are three types of gradient descent learning algorithms:

1. Batch gradient descent,
2. Stochastic gradient descent
3. Mini-batch gradient descent.



BATCH GRADIENT DESCENT: Batch gradient descent, also known as vanilla gradient descent, calculates the error for each example within the training dataset. Still, the model is not changed until every training sample has been assessed. The entire procedure is referred to as a cycle and a training epoch.

Some benefits of batch are its computational efficiency, which produces a stable error gradient and a stable convergence. Some drawbacks are that the stable error gradient can sometimes result in a state of convergence that isn't the best the model can achieve. It also requires the entire training dataset to be in memory and available to the algorithm.

Advantages

1. Fewer model updates mean that this variant of the steepest descent method is more computationally efficient than the stochastic gradient descent method.
2. Reducing the update frequency provides a more stable error gradient and a more stable convergence for some problems.
3. Separating forecast error calculations and model updates provides a parallel processing-based algorithm implementation.

Disadvantages

1. A more stable error gradient can cause the model to prematurely converge to a suboptimal set of parameters.
2. End-of-training epoch updates require the additional complexity of accumulating prediction errors across all training examples.
3. The batch gradient descent method typically requires the entire training dataset in memory and is implemented for use in the algorithm.
4. Large datasets can result in very slow model updates or training speeds.
5. Slow and require more computational power.

STOCHASTIC GRADIENT DESCENT: By contrast, stochastic gradient descent (SGD) changes the parameters for each training sample one at a time for each training example in the dataset. Depending on the issue, this can make SGD faster than batch gradient descent. One benefit is that the regular updates give us a fairly accurate idea of the rate of improvement.

However, the batch approach is less computationally expensive than the frequent updates. The frequency of such updates can also produce noisy gradients, which could cause the error rate to fluctuate rather than gradually go down.

Advantages

1. You can instantly see your model's performance and improvement rates with frequent updates.
2. This variant of the steepest descent method is probably the easiest to understand and implement, especially for beginners.
3. Increasing the frequency of model updates will allow you to learn more about some issues faster.
4. The noisy update process allows the model to avoid local minima (e.g., premature convergence).
5. Faster and require less computational power.
6. Suitable for the larger dataset.

Disadvantages

1. Frequent model updates are more computationally intensive than other steepest descent configurations, and it takes considerable time to train the model with large datasets.
2. Frequent updates can result in noisy gradient signals. This can result in model parameters and cause errors to fly around (more variance across the training epoch).
3. A noisy learning process along the error gradient can also make it difficult for the algorithm to commit to the model's minimum error.

MINI-BATCH GRADIENT DESCENT: Since mini-batch gradient descent combines the ideas of batch gradient descent with SGD, it is the preferred technique. It divides the training dataset into manageable groups and updates each separately. This strikes a balance between batch gradient descent's effectiveness and stochastic gradient descent's durability.

Mini-batch sizes typically range from 50 to 256, although, like with other machine learning techniques, there is no set standard because it depends on the application. The most popular kind in deep learning, this method is used when training a neural network.

Advantages

1. The model is updated more frequently than the stack gradient descent method, allowing for more robust convergence and avoiding local minima.
2. Batch updates provide a more computationally efficient process than stochastic gradient descent.
3. Batch processing allows for both the efficiency of not having all the training data in memory and implementing the algorithm.

Disadvantages

1. Mini-batch requires additional hyper parameters "mini-batch size" to be set for the learning algorithm.
2. Error information should be accumulated over a mini-batch of training samples, such as batch gradient descent.
3. it will generate complex functions.

Challenges with gradient descent

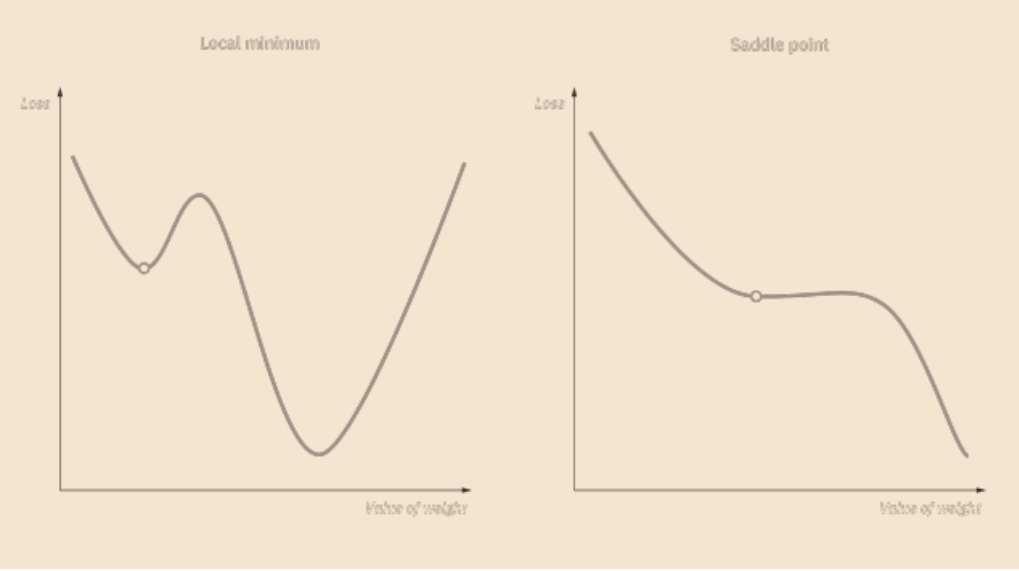
While gradient descent is the most common approach for optimization problems, it does come with its own set of challenges. Some of them include:

Local minima and saddle points: For convex problems, gradient descent can find the global minimum with ease, but as non convex problems emerge, gradient descent can struggle to find the global minimum, where the model achieves the best results.

When the slope of the cost function is at or close to zero, the model stops learning. A few scenarios beyond the global minimum can also yield this slope, which are local minima and saddle points. Local minima mimic the shape of a global minimum, where the slope of the cost function increases on either side of the current point. However, with saddle points, the negative gradient only exists on one side of the point, reaching a local maximum on one side and a local minimum on the other. Its name inspired by that of a horse's saddle. Noisy gradients can help the gradient escape local minimums and saddle points.

Vanishing and Exploding Gradients: In deeper neural networks, particular recurrent neural networks, we can also encounter two other problems when the model is trained with gradient descent and backpropagation.

- **Vanishing gradients:** This occurs when the gradient is too small. As we move backwards during backpropagation, the gradient continues to become smaller, causing the earlier layers in the network to learn more slowly than later layers. When this happens, the weight parameters update until they become insignificant—i.e. 0—resulting in an algorithm that is no longer learning.
- **Exploding gradients:** This happens when the gradient is too large, creating an unstable model. In this case, the model weights will grow too large, and they will eventually be represented as NaN. One solution to this issue is to leverage a dimensionality reduction technique, which can help to minimize complexity within the model.



12) Explain about Architecture Design?

A) Architecture Design: The key design consideration for neural networks is determining the architecture. The word **architecture** refers to the overall structure of the network: how many units it should have and how these units should be connected to each other.

Most neural networks are organized into groups of units called layers. Most neural network architectures arrange these layers in a chain structure, with each layer being a function of the layer that preceded it. In this structure, the first layer is given by

$$\boldsymbol{h}^{(1)} = g^{(1)\,\mathrm{T}}\boldsymbol{W}^{(1)T}\boldsymbol{x} + \boldsymbol{b}^{(1)\mathrm{T}},$$

the second layer is given by

$$\boldsymbol{h}^{(2)} = g^{(2)\,T}\boldsymbol{W}^{(2)T}\boldsymbol{h}^{1)} + \boldsymbol{b}^{(2)T},$$

and so on.

In these chain-based architectures, the main architectural considerations are to choose the depth of the network and the width of each layer. A network with even one hidden layer is sufficient to fit the training set.

Deeper networks often are able to use far fewer units per layer and far fewer parameters and often generalize to the test set, but are also often harder to optimize. The ideal network architecture for a task must be found via experimentation guided by monitoring the validation set error.

Universal Approximation Properties and Depth

Feedforward networks with hidden layers provide a universal approximation framework. The **universal approximation theorem** states that a feedforward network with a linear output layer and at least one hidden layer with any "squashing" activation function (such as the logistic sigmoid activation function) can approximate any Borel measurable function from one finite-dimensional space to another with any desired non-zero amount of error, provided that the network is given enough hidden units.

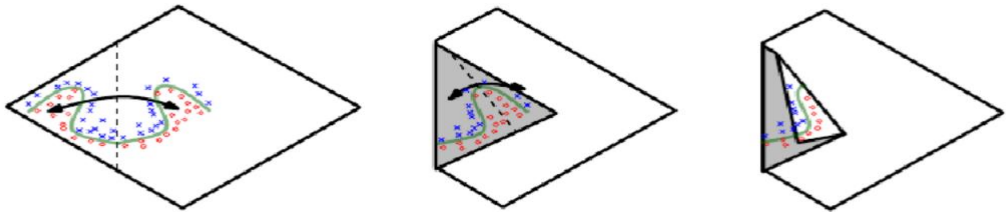
The derivatives of the feedforward network can also approximate the derivatives of the function arbitrarily well

The universal approximation theorem means that regardless of what function we are trying to learn, we know that a large MLP will be able to *represent* this function.

The universal approximation theorem says that there exists a network large enough to achieve any degree of accuracy we desire, but the theorem does not say how large this network will be

The universal approximation theorem says that there exists a network large enough to achieve any degree of accuracy we desire, but the theorem does not say how large this network will be. There exist families of functions which can be approximated efficiently by an architecture with depth greater than some value d, but which require a much larger model if depth is restricted to be less than or equal to d.

The below picture illustrates how a network with absolute value rectification creates mirror images of the function computed on top of some hidden unit, with respect to the input of that hidden unit. Each hidden unit specifies where to fold the input space in order to create mirror responses (on both sides of the absolute value nonlinearity). By composing these folding operations, we obtain an exponentially large number of piecewise linear regions which can capture all kinds of regular (e.g., repeating) patterns.



Another key consideration of architecture design is exactly how to connect a pair of layers to each other. In the default neural network layer described by a linear transformation via a matrix $\boldsymbol{W}$, every input unit is connected to every output unit. Many specialized networks in the chapters ahead have fewer connections, so that each unit in the input layer is connected to only a small subset of units in the output layer. These strategies for reducing the number of connections reduce the number of parameters and the amount of computation required to evaluate the network, but are often highly problem-dependent.

13) Explain about Regularization in DNN?

Regularization: we can define regularization as "any modification we make to a learning algorithm that is intended to reduce its generalization error but not its training error." There are many regularization strategies. In the context of deep learning, most regularization strategies are based on regularizing estimators. Regularization of an estimator works by trading increased bias for reduced variance. An effective regularizer is one that makes a profitable trade, reducing variance significantly while not overly increasing the bias.

Types of regularization techniques: Regularization is a technique used to address overfitting by directly changing the architecture of the model by modifying the model's training process. The following are the commonly used regularization techniques:

1. L2 regularization
2. L1 regularization
3. Dropout regularization

L2 regularization: According to regression analysis, L2 regularization is also called ridge regression. In this type of regularization, the squared magnitude of the coefficients or weights multiplied with a regularizer term is added to the loss or cost function. L2 regression can be represented with the following mathematical equation.

Loss:

$$\underbrace{\sum_{i=1}^{n}\left(y_i - \sum_{j=1}^{p} x_{ij} b_j\right)^2}_{Loss\ term} + \underbrace{\lambda \cdot \left(\sum_{j=1}^{p} b_j\right)^2}_{Regularizer\ term}$$

In the above equation,

$y_i \ - \ labels$
$x_{ij} \ - \ features$
$b_j \ - \ weights$
$\lambda \ - \ regularizer\, term\ (lambda)$
$i \ - \ no\, of\, training\, samples$

We can see that a fraction of the sum of squared values of weights is added to the loss function. Thus, when gradient descent is applied on loss, the weight update seems to be consistent by giving almost equal emphasis on all features.

- Lambda is the hyperparameter that is tuned to prevent overfitting i.e. penalize the insignificant weights by forcing them to be small but not zero.
- L2 regularization works best when all the weights are roughly of the same size, i.e., input features are of the same range.
- This technique also helps the model to learn more complex patterns from data without overfitting easily.

L1 regularization: L1 regularization is also referred to as lasso regression. In this type of regularization, the absolute value of the magnitude of coefficients or weights multiplied with a regularizer term is added to the loss or cost function. It can be represented with the following equation.

Loss:

$$\underbrace{\sum_{i=1}^{n}\left(y_i - \sum_{j=1}^{p} x_{ij}b_j\right)^2}_{Loss\ Term} + \underbrace{\lambda \cdot \sum_{j=1}^{p}|b_j|}_{Regularizer\ term}$$

In the above equation,

$$y_i \ - \ labels$$
$$x_{ij} \ - \ features$$
$$b_j \ - \ weights$$
$$\lambda \ - \ regularizer\ term\ (lambda)$$
$$i \ - \ no\ of\ training\ samples$$

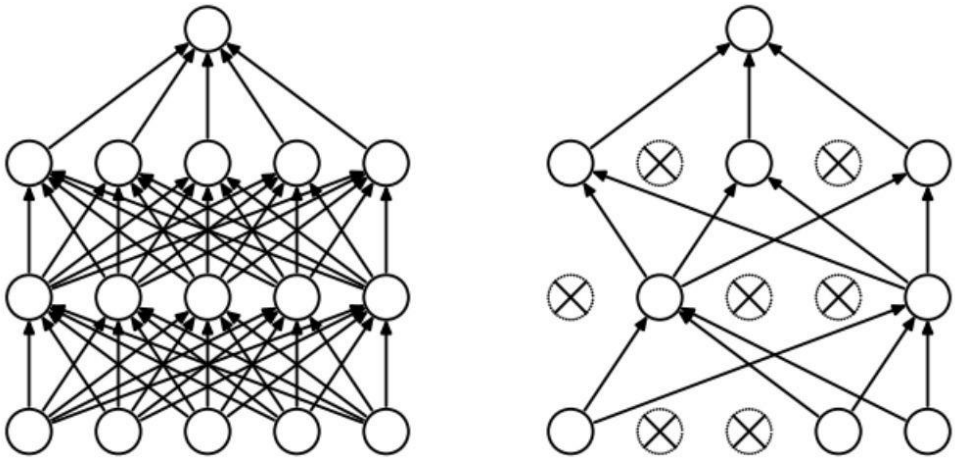
A fraction of the sum of absolute values of weights to the loss function is added in the L1 regularization. In this way, you will be able to eliminate some coefficients with lesser values by pushing those values towards 0. You can observe the following by using L1 regularization:

- Since the L1 regularization adds an absolute value as a penalty to the cost function, the feature selection will be done by retaining only some important features and eliminating the lower or unimportant features.
- This technique is also robust to outliers, i.e., the model will be able to easily learn about outliers in the dataset.
- This technique will not be able to learn complex patterns from the input data.

Dropout regularization: Dropout regularization is the technique in which some of the neurons are randomly disabled during the training such that the model can extract more useful robust features from the model. This prevents overfitting. You can see the dropout regularization in the following diagram:

- In figure (a), the neural network is fully connected. If all the neurons are trained with the entire training dataset, some neurons might memorize the patterns occurring in training data. This leads to overfitting since the model is not generalizing well.

- In figure (b), the neural network is sparsely connected, i.e., only some neurons are active during the model training. This forces the neurons to extract robust features/patterns from training data to prevent overfitting.



(a) Standard Neural Net (b) After applying dropout.

The following are the characteristics of dropout regularization:

- Dropout randomly disables some percent of neurons in each layer. So for every epoch, different neurons will be dropped leading to effective learning.
- Dropout is applied by specifying the 'p' values, which is the fraction of neurons to be dropped.
- Dropout reduces the dependencies of neurons on other neurons, resulting in more robust model behavior.
- Dropout is applied only during the model training phase and is not applied during the inference phase.
- When the model receives complete data during the inference time, you need to scale the layer outputs 'x' by 'p' such that only some parts of data will be sent to the next layer. This is because the layers have seen less amount of data as specified by dropout.

These are some of the most popular regularization techniques that are used to reduce overfitting during model training. They can be applied according to the use case or dataset being considered for more accurate model performance on the testing data.

Need of Regularisation: In a general learning algorithm, the dataset is divided into a **training set** and a **test set.** After each epoch of the algorithm, the parameters are updated accordingly after understanding the dataset. Finally, this trained model is applied to the test set.

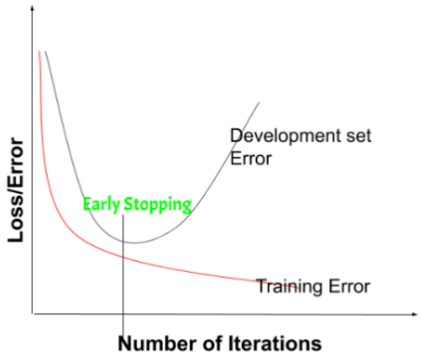
Generally, the training set error will be less compared to the test set error. This is because of overfitting whereby the algorithm memorizes the training data and produces the right results on the training set. So, the model becomes highly exclusive to the training set and fails to produce accurate results for other datasets including the test set. Regularization techniques are used in such situations to **reduce overfitting** and **increase the model's performance** on any general dataset.

14) What is Early Stopping?

In Regularization by Early Stopping, we stop training the model when the performance on the validation set is getting worse- increasing loss decreasing accuracy, or poorer scores of the scoring metric. By

plotting the error on the training dataset and the validation dataset together, both the errors decrease with a number of iterations until the point where the model starts to overfit. After this point, the training error still decreases but the validation error increases.

So, even if training is continued after this point, early stopping essentially returns the set of parameters that were used at this point and so is equivalent to stopping training at that point. So, the final parameters returned will enable the model to have low variance and better generalization. The model at the time the training is stopped will have a better generalization performance than the model with the least training error.



Early stopping can be thought of as **implicit regularization**, contrary to regularization via weight decay. This method is also efficient since it requires less amount of training data, which is not always available. Due to this fact, early stopping requires lesser time for training compared to other regularization methods. Repeating the early stopping process many times may result in the model overfitting the validation dataset, just as similar as overfitting occurs in the case of training data.

The number of iterations(i.e. epoch) taken to train the model can be considered a **hyperparameter**. Then the model has to find an optimum value for this hyperparameter (by hyperparameter tuning) for the best performance of the learning model.

Benefits of Early Stopping:

- Helps in reducing overfitting
- It improves generalisation
- It requires less amount of training data
- Takes less time compared to other regularisation models
- It is simple to implement

Limitations of Early Stopping:

- If the model stops too early, there might be risk of underfitting
- It may not be beneficial for all types of models
- If validation set is not chosen properly, it may not lead to the most optimal stopping

To summarize, early stopping can be best used to prevent overfitting of the model, and saving resources. It would give best results if taken care of few things like – parameter tuning, preventing the model from overfitting, and ensuring that the model learns enough from the data.

Algorithm : The early stopping meta-algorithm for determining the best amount of

time to train. This meta-algorithm is a general strategy that works well with a variety of training algorithms and ways of quantifying error on the validation set.

Let n be the number of steps between evaluations.
Let p be the "patience," the number of times to observe worsening validation set error before giving up.
Let $\boldsymbol{\theta}_o$ be the initial parameters.
$\boldsymbol{\theta} \rightarrow \boldsymbol{\theta}_o$

$i \rightarrow 0$

$j \rightarrow 0$ $v \rightarrow$
∞ $\boldsymbol{\theta}^* \rightarrow \boldsymbol{\theta}$
$i^* \rightarrow i$

while $j < p$ **do**

Update $\boldsymbol{\theta}$ by running the training algorithm for n steps.
$i \rightarrow i + n$

$v^T \rightarrow$ ValidationSetError($\boldsymbol{\theta}$)
if $v^T < v$ **then**

$j \rightarrow 0$ $\boldsymbol{\theta}^* \rightarrow$
$\boldsymbol{\theta}$ $i^* \rightarrow i$ v
$\rightarrow v^T$

else

$j \rightarrow j + 1$

end if end while

Best parameters are $\boldsymbol{\theta}^*$, best number of training steps is i^*

15) What Are Optimizers in Deep Learning? Explain about Adagard and Adam Optimizers?

In deep learning, optimizers are algorithms that adjust the model's parameters during training to minimize a loss function. They enable neural networks to learn from data by iteratively updating weights and biases Each optimizer has specific update rules, learning rates, and momentum to find optimal model parameters for improved performance.

An optimizer is a function or an algorithm that adjusts the attributes of the neural network, such as weights and learning rates. Thus, it helps in reducing the overall loss and improving accuracy. The problem of choosing the right weights for the model is a daunting task, as a deep learning model generally consists of millions of parameters. This various deep-learning optimizers are Gradient Descent, Stochastic Gradient Descent, Stochastic Gradient descent with momentum, Mini-Batch Gradient Descent, Adagrad, RMSProp, AdaDelta, and Adam.

Adagrad (Adaptive Gradient Descent) Deep Learning Optimizer: The adaptive gradient descent algorithm is slightly different from other gradient descent algorithms. This is because it uses different

learning rates for each iteration. The change in learning rate depends upon the difference in the parameters during training. The more the parameters get changed, the more minor the learning rate changes. This modification is highly beneficial because real-world datasets contain sparse as well as dense features. So it is unfair to have the same value of learning rate for all the features. The Adagrad algorithm uses the below formula to update the weights. Here the alpha(t) denotes the different learning rates at each iteration, n is a constant, and E is a small positive to avoid division by 0.

$$\mathrm{w_t} = \mathrm{w_{t-1}} - \eta'_t \frac{\partial \mathrm{L}}{\partial \mathrm{w(t-1)}} \qquad \eta'_t = \frac{\eta}{\mathrm{sqrt}(\alpha_t + \epsilon)}$$

The benefit of using Adagrad is that it abolishes the need to modify the learning rate manually. It is more reliable than gradient descent algorithms and their variants, and it reaches convergence at a higher speed.

One downside of the AdaGrad optimizer is that it decreases the learning rate aggressively and monotonically. There might be a point when the learning rate becomes extremely small. This is because the squared gradients in the denominator keep accumulating, and thus the denominator part keeps on increasing. Due to small learning rates, the model eventually becomes unable to acquire more knowledge, and hence the accuracy of the model is compromised.

Benefits of using AdaGrad

- **Easy to use**– It's a reasonably straightforward optimization technique and may be applied to various models.
- **No need for manual**– There is no need to manually tune hyperparameters since this optimization method automatically adjusts the learning rate for each parameter.
- **Adaptive learning rate**– Modifies the learning rate for each parameter depending on the parameter's past gradients. This implies that for parameters with big gradients, the learning rate is lowered, while for parameters with small gradients, the learning rate is raised, allowing the algorithm to converge quicker and prevent overshooting the ideal solution.
- **Adaptability to noisy data**– This method provides the ability to smooth out the impacts of noisy data by assigning lesser learning rates to parameters with strong gradients owing to noisy input.
- **Handling sparse data efficiently**– It is particularly good at dealing with sparse data, which is prevalent in NLP and recommendation systems. This is performed by giving sparse parameters faster learning rates, which may speed convergence.

Adam Optimizer in Deep Learning: Adam optimizer, short for Adaptive Moment Estimation optimizer, is an optimization algorithm commonly used in deep learning. It is an extension of the stochastic gradient descent (SGD) algorithm and is designed to update the weights of a neural network during training.

The name "Adam" is derived from "adaptive moment estimation," highlighting its ability to adaptively adjust the learning rate for each network weight individually. Unlike SGD, which maintains a single learning rate throughout training, Adam optimizer dynamically computes individual learning rates based on the past gradients and their second moments.

Adam optimizer is an optimization algorithm that extends SGD by dynamically adjusting learning rates based on individual weights. It combines the features of AdaGrad and RMSProp to provide efficient and adaptive updates to the network weights during deep learning training.

Adam Optimizer Formula: The adam optimizer has several benefits, due to which it is used widely. It is adapted as a benchmark for deep learning papers and recommended as a default optimization algorithm. Moreover, the algorithm is straightforward to implement, has a faster running time, low memory requirements, and requires less tuning than any other optimization algorithm.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \left[\frac{\delta L}{\delta w_t} \right] v_t = \beta_2 v_{t-1} + (1 - \beta_2) \left[\frac{\delta L}{\delta w_t} \right]^2$$

The above formula represents the working of adam optimizer. Here B1 and B2 represent the decay rate of the average of the gradients.

Advantages:

1. The method is too fast and converges rapidly.
2. Rectifies vanishing learning rate, high variance.

Disadvantages:

Computationally costly.